

Rust - Simple script

Script simple ("Hello World")

Créer un répertoire de projet et y insérer le fichier `main.rs` contenant :

```
fn main() {  
    println!("Hello, world !");  
}
```

Puis lancer les commandes suivantes :

- Compilation : `rustc main.rs` => [Rust](#) produit un binaire exécutable (sous [Linux](#), le fichier `main`).
- Exécution : `./main`
- Formatage : `rustfmt main.rs` (si non réalisé par le *linter* de l'IDE)

[Rust](#) est un langage à *compilation anticipée*, ce qui veut dire que nous pouvons compiler le programme et le donner à quelqu'un d'autre, et il peut l'exécuter sans avoir [Rust](#) d'installé.

Si `rustc` peut suffire pour compiler de simples scripts, des projets plus élaborés nécessiteront l'emploi de l'outil [Cargo](#).

Structure du script

```
fn main() {  
  
}
```

- `fn` définit une [fonction](#).
- La fonction `main` (ici, sans paramètre et sans valeur de retour) = fonction spéciale, car il s'agit toujours du premier code exécuté dans un programme [Rust](#).
- Corps de la fonction placé entre accolades (accolade ouvrante sur la même ligne que la définition = bonne pratique).

```
    println!("Hello, world!");
```

- Indentation avec 4 espaces et non une tabulation.
- `println!` = macro (cf. chap. 19 du [Rust Book](#)). `!` signifie que l'on utilise une macro.
- `"Hello, world:"` = argument de la macro.
- `;` indique la fin d'une instruction.